

Resumen Gestion de Proyectos

Metodología de sistemas de información

Es el punto medio entre el enfoque sistémico y las metodologías para aplicar buenas prácticas ingenieriles.

Una metodología es un conjunto de pasos en orden determinado que permiten el logro de un objetivo.

Una metodología, para construir un producto, precisa de 5 pasos.

- Decidir **qué** hacer
- Decidir **cómo** hacerlo
- **Hacerlo**
- **Probar** el resultado
- **Usar** el producto

Estudio de factibilidad

Tiene por objetivo elegir el camino apropiado según una serie de criterios ya establecidos en función de aspectos económicos, técnicos, legales, etc.

La equifinalidad hace referencia al proceso de poder llegar al mismo destino por varios caminos.

La ponderación de las distintas alternativas se realizan conjuntamente entre el profesional y el cliente.

Después de analizar las diferentes alternativas se selecciona la mejor propuesta, y esta recibe el nombre de “**Alternativa Solución**”.

La alternativa solución no es necesariamente desarrollar software; puede estar compuesto por diferentes elementos, entre ellos:

- Comprar, alquilar o adaptar software.
- Capacitación del personal.

Hay distintos tipos de estudios de factibilidad.

- **Técnico:** se evalúa si el producto es técnicamente implementable (funcionalidad, rendimiento, restricciones). Se analiza software y hardware.
- **Operativo:** se valora la capacidad de la organización para poder operar correctamente el artefacto. También se observa que capacitaciones se requieren para lograrlo.
- **Económico:** se pondera si realmente es necesaria la implementación del nuevo producto, de acuerdo a la relación costo / beneficio.
- **Legal:** se evalúa la normativa vigente relacionada al sistema.

Mantenimiento

Esta etapa busca asegurar la validez del producto desarrollado, y su persistencia en el tiempo.

La solución no puede permanecer inalterable, ya que el entorno cambia y se deben poder otorgar respuestas de ajuste.

Tipos de acciones de mantenimiento.

- **Correctivas:** corrigen desvíos que no fueron detectados en la etapa de pruebas.
- **Perfectivas:** perfeccionan la solución, dando respuesta a nuevos requerimientos u optimizando soluciones previas.
- **Adaptativas:** se adapta la solución a cambios ajenos pero que afectan el funcionamiento (nuevo sistema operativo, nuevas leyes, etc.)

La persistencia es lo que hace que un producto sea útil: robusto, íntegro y resistente que tolera el uso y el desgaste.

Pruebas

El objetivo es encontrar la mayor cantidad de fallas y realizar las correcciones necesarias, acercándose lo más posible a los estándares de calidad deseados.

Una buena prueba debe tener las siguientes características.

- Alta probabilidad de encontrar un error.
- No debe tener el mismo propósito que otra prueba, y en caso de que lo tenga, debe ser la mejor y descartar las otras.
- No debe ser demasiado simple ni demasiado compleja.

Estrategias de pruebas.

- **Unidad:** se analiza el funcionamiento de cada unidad a fondo. Se utilizan muchas técnicas que ejercitan rutas específicas que aseguran una cobertura completa y máxima detección de errores.
- **Integración:** pretende ver las relaciones y evitar conflictos entre los módulos definidos.
- **Validación:** proporciona garantía final de que el software cumple con todos los requerimientos (informativos, funcionales, rendimiento, etc.).
- **Sistema:** verifica que todos los elementos se mezclan de manera adecuada y se logra el funcionamiento / rendimiento global del sistema.
- **Aceptación:** se hace participar al usuario en el proyecto.

Técnicas de pruebas.

- **Caja negra:** se validan entradas contra salidas, dejando de lado el proceso interno.
- **Caja blanca:** se valida la entrada y las rutas lógicas que llevan a la salida.

Las **pruebas de caja negra** son las que se llevan a cabo en la interfaz del software. No se preocupan por la estructura lógica interna, se enfocan en los requerimientos funcionales. No son una alternativa a las pruebas de caja blanca, ya que se pueden descubrir errores diferentes, y a diferencia de estas últimas, las pruebas de caja negra se aplican en las últimas etapas de prueba.

Se utilizan para encontrar errores de las siguientes categorías.

- Funciones incorrectas o faltantes.
- Errores de interfaz.
- Errores en las estructuras de datos o acceso a bases de datos externas.
- Errores de comportamiento o rendimiento.
- Errores de inicialización y terminación.

La partición de equivalencia es un método que divide el dominio de entrada (tipo de dato) para calcular los casos de prueba.

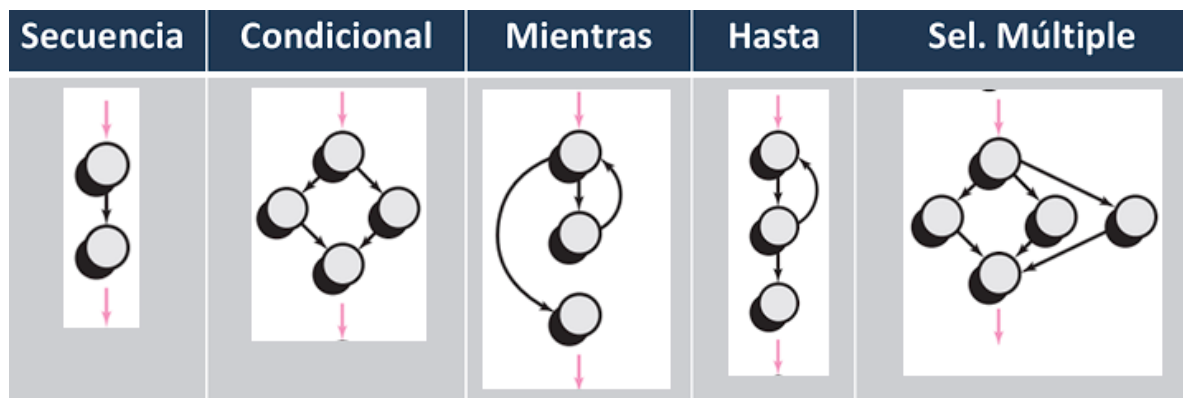
Se evalúan las clases de equivalencia según las siguientes condiciones de entrada.

- Si especifica un rango o requiere un valor específico: una clase de equivalencia válida y dos inválidas.

- Si especifica un miembro de un conjunto o es boolean: una clase de equivalencia válida y una inválida

Las **pruebas de caja blanca** permiten generar casos de prueba que garanticen que todas las rutas independientes se revisaron al menos una vez, incluyendo decisiones lógicas como verdadero / falso, y ejecuten todos los bucles en y dentro de sus fronteras operativas.

Se utilizan para calcular la complejidad ciclomática (medida de complejidad lógica).



Los círculos son nodos, y las flechas aristas. Las áreas acotadas por estos, se llaman regiones, y la parte fuera del área también se considera una región.

Cuando aparece una condición compuesta por operadores booleanos (or, and, etc), se crea un nodo separado para cada una de las condiciones. Los nodos que contienen una condición se llaman nodos predicados, y tienen 2 o más aristas que salen de él.

Una ruta independiente es cualquiera que introduce al menos un nuevo conjunto de enunciados o una nueva condición. Cada ruta independiente debe moverse a lo largo de al menos una arista que no se haya recorrido.

El método **simplificado** comienza seleccionando el camino más corto de principio a fin, y luego va buscando segmentos no recorridos hasta cumplir con el número de rutas recorridos. Este método tiende a recorrer primero los caminos de excepción, errores y salidas de emergencia.

Para saber cuántas rutas buscar, se realizan los cálculos de complejidad ciclomática.

1. $V(G) = \text{Cant. de Regiones}$
2. $V(G) = \text{Cant. Aristas} - \text{Cant. Nodos} + 2$
3. $V(G) = \text{Cant. Nodos Predicados} + 1$

Las 4 P's

Las 4 P's hacen referencia al marco de trabajo fundamental para lograr buenos resultados en el **proceso** de desarrollo de un **producto** en el contexto de un **proyecto** de software.

Las 4 P son **personas**, **producto**, **proceso** y **proyecto**.

Personas

Son los partícipes del proyecto de software. Abarca a gerentes ejecutivos, gerentes de proyecto, profesionales, clientes y usuarios finales, entre otros.

Un buen líder / gerente de proyecto debe tener la capacidad de motivar (con un fin, no una simple arenga) a su personal, debe tener la habilidad para organizar y moldear procesos (ya existentes o nuevos) que permitan que el concepto sea un producto final, y debe tener la capacidad de innovar y aportar ideas, que el personal sea incentivado para sentirse creativo, siempre y cuando se mantengan dentro de las fronteras establecidas.

Es importante que los miembros del equipo:

- Tengan confianza entre sí.
- Esten distribuidos correctamente en base a sus habilidades.

Se debe evitar la toxicidad de equipo (atmósfera de trabajo frenético, alta frustración, definición poco clara de roles, etc).

Producto

Es el software, se sustenta en otros productos generados por profesionales de la rama. Debe ser desarrollado por profesionales (ingenieros en sistemas / software, arquitectos de software, diseñadores, analistas, programadores).

Los pasos para obtener el producto dependen de la metodología que se elija para llevarlo a cabo.

Pueden ser **lineales, incrementales, iterativas o evolutivas, y ágiles.**

Como producto final se obtiene:

- Programas, archivos ejecutables o código fuente.
- Diseño lógico de datos.
- Documentación: especificaciones, modelos, diagramas, manuales, etc.

A diferencia de la curva de fallas del hardware, la curva de fallas del software indica que el producto no se desgasta, pero si se deteriora. Lo ideal es que se le dé importancia a la persistencia del producto en el tiempo.

Proceso

Es el marco de trabajo que define el enfoque que se toma cuando el software es tratado por la ingeniería.

El proceso de software es una estructura para las actividades, acciones y tareas que se requieren para construir un software de alta calidad.

Una **actividad** busca lograr un objetivo amplio, una **acción** es un conjunto de tareas que producen un producto importante del trabajo, y una **tarea** se centra en un objetivo pequeño pero bien definido.

Una estructura de la ingeniería de software se define en 5 actividades estructurales: **comunicación, planeación, modelado, construcción y despliegue.**

Se organizan de forma diferente según el modelo a utilizar.

LINEAL	INCREMENTAL	ITERATIVO o EVOLUTIVO	ITERATIVO E INCREMENTAL	AGILES
Cascada (waterfall)	Incremental	Prototipos	Proceso Unificado	Scrum
	Desarrollo Rápido de Aplicaciones (DRA)	Espiral	Iconix	Kanban
				XP

El modelo de cascada se destaca por tener escasa participación del cliente. Debe tener una especificación de requerimientos completa al comienzo del proyecto. No hay trabajo ni pruebas en paralelo, por ende la detección de errores es tardía. Su proceso es secuencial y no se puede volver a etapas anteriores.

El modelo incremental tiene un proceso secuencial pero que abarca funcionalidades específicas, es similar al modelo en cascada pero en este caso el cliente puede ver una versión ejecutable del software al final de cada incremento.

El modelo espiral actúa de forma diferente a los dos mencionados anteriormente, ya que cada vuelta del espiral pasa por los 5 procesos necesarios, y también se incorpora el concepto de riesgo y se lo gestiona. Es un enfoque de crecimiento cíclico, que permite refinar el software en base a la retroalimentación.

El proceso unificado nace a partir del uso de la tecnología UML, ya que se necesitaba una estructura del proceso que guiará a los equipos que apliquen esta misma. Hay refinamiento e integración de funcionalidades anteriores hasta llegar a la versión deseada por el cliente con cada incremento. Está centrado en la arquitectura, y se dirige por casos de usos. Ambos mencionados evolucionan en paralelo.

El proceso ágil tiene una colaboración continua con el cliente (retroalimentación). Prioriza el software funciona por sobre la documentación exhaustiva, responde al cambio, permite la entrega rápida.

El manifiesto del proceso ágil abarca 4 puntos:

- Individuos e interacciones > procesos y herramientas.
- Software funcionando > documentación exhaustiva.
- Colaboración con el cliente > negociaciones contractuales.
- Responder al cambio > seguir un plan.

Si bien los elementos de la derecha tienen valor, los de la izquierda son más valiosos.

Proceso

Dentro de los procesos encontramos los siguientes:

- **Predictivos:** se tiene conocimiento total de los requerimientos, por lo cual es posible realizar una planificación completa. Se realiza una única entrega al final del ciclo (ejemplo: construcción de casa).
- **Iterativos:** no se tiene conocimiento total de los requerimientos. A medida que se itera y se entrega una funcionalidad, podremos obtener retroalimentación y realizar correcciones / mejoras para futuras iteraciones, logrando así el perfeccionamiento. Cada iteración no es necesariamente una entrega definitiva de las funcionalidades desarrolladas hasta el momento. (ejemplo: construcción de un robot).
- **Incrementales:** no se tiene conocimiento total de los requerimientos. En cada iteración se realiza una entrega funcional real que se va adicionando al producto incrementando las funcionalidades.
- **Ágiles:** no se tiene un conocimiento total de los requerimientos. Con cada iteración se realiza una entrega funcional real pero a su vez se aporta la retroalimentación para la próxima iteración. Combina iterativo e incremental.

En el proceso de desarrollo de software es importante entender el contexto para seleccionar el modelo de proceso apropiado (ejemplo, en una aplicación educativa se valoran los comentarios de los usuarios y la necesidad de actualizaciones frecuentes basadas en la participación de los estudiantes, o un sistema bancario, se deben evaluar requisitos de cumplimiento de seguridad, precisión de transacciones, etc.).

Proyecto

Es un esfuerzo temporal emprendido para crear un producto único.

Todo proyecto se caracteriza por tener:

- Alcance y entregables definidos, objetivos específicos y medibles definidos por los criterios de éxito previamente establecidos.
- Cronograma fijo con fechas de inicio y fin.
- Recursos asignados, asignación y gestión cuidadosa.
- Expectativas de calidad y criterios de aceptación.
- Gestión activa, los proyectos requieren una planificación, seguimiento, y control continuo para garantizar una entrega exitosa.

Un proyecto es diferente a tareas rutinarias ya que sus necesidades son más críticas en términos de planificación, ejecución y gestión.

Un proyecto es temporal, creando resultados únicos. Requiere planificación exhaustiva, con mayor riesgo y cambios frecuentes, recursos variables (ejemplo: crear una aplicación móvil para un festival, o una plataforma para una universidad).

Una tarea rutinaria es continua y repetitiva, produciendo resultados similares. Sigue procedimientos ya establecidos, siendo un proceso predecible, con una necesidad de recursos constante (ejemplo: facturación mensual de clientes, atención al cliente, copias de seguridad diarias).

Planificación

El objetivo de la planificación del proyecto de software es proporcionar un marco conceptual que permita al gerente hacer estimaciones razonables de recursos, costo y calendario. Estas estimaciones deben abarcar los escenarios de mejor y peor caso.

La planificación de un proyecto involucra un conjunto de tareas:

- Establecer el ámbito del proyecto y su factibilidad.
- Analizar los riesgos.
- Definir los recursos requeridos: personal, componentes de software reutilizables, entorno de desarrollo a utilizar, etc.
- Estimar costos y esfuerzo.
- Desarrollar el calendario del proyecto.

Para administrar un proyecto de software exitoso, se debe tener en cuenta todo lo que pueda salir mal (ejemplo: el personal no entiende las necesidades del cliente, las fechas límite son irreales, los usuarios son resistentes, etc).

El desarrollo planificado sigue una metodología estructurada con requisitos claros, documentación y revisiones periódicas, mientras que en el desarrollo improvisado el equipo trabaja sin procesos formados, confiando en la experiencia individual.

Riesgo de Software

Un proyecto de software es complejo, por ende hay un sinnúmero de situaciones que pueden salir mal. Un riesgo es un problema potencial.

Los riesgos involucran dos características:

- **Incertidumbre:** el riesgo puede o no ocurrir.
- **Pérdida:** si el riesgo se vuelve realidad, ocurrirán consecuencias no deseadas.

En base a los riesgos se pueden tomar dos estrategias:

- **Reactivas:** no hay preocupación por el problema hasta que ocurre.
- **Proactivas:** se identifican riesgos potenciales, se establecen planes para gestionar el riesgo y se desarrollan planes de contingencia en caso de que no se pueda evitar el riesgo.

Se pueden identificar 3 tipos de riesgo:

- **Riesgos de proyecto:** presupuesto, personal, recursos, etc.
- **Riesgos técnicos:** diseño, implementación, mantenimiento, etc.
- **Riesgos empresariales:** producto no requerido por el mercado, etc.

También se pueden categorizar en riesgos conocidos (se descubren en el desarrollo), riesgos predecibles (se conocen a partir de la experiencia de proyectos anteriores), y riesgos impredecibles (extremadamente difíciles de identificar por adelantado).

Una tabla de riesgos proporciona una técnica simple para la proyección de riesgos. Consiste en 4 columnas: descripción de riesgo, categoría (ej: riesgo empresarial), probabilidad de ocurrencia, y valor del impacto (1- catastrófico, 2-critico, 3-marginal, 4-despreciable). Una vez identificado los posibles riesgos, se ordena la lista según probabilidad de ocurrencia e impacto en forma descendente. Se define una línea de corte y todo lo que esté por encima debe tener un plan de gestión de riesgo, monitoreo, y plan de contingencia.

Roles

En un equipo de desarrollo de software se requiere la coordinación entre especialistas con habilidades y responsabilidades complementarias.

Una definición clara de roles ayuda a:

- Establecer responsabilidades.
- Mejorar la comunicación.
- Asegurar que todos los aspectos del desarrollo estén cubiertos.
- Reducir la superposición y confusión.

Cada rol aporta una experiencia y perspectiva única para crear un producto cohesivo y exitoso.

El **desarrollador** tiene:

- **Responsabilidades principales:**
 - Escribir código limpio y fácil de mantener.
 - Implementar funciones de acuerdo a especificaciones.
 - Traducir diseños técnicos en software funcional.
 - Realizar pruebas unitarias en componentes desarrollados.
- **Habilidades requeridas:**
 - Dominio de lenguajes de programación.
 - Habilidades para resolver problemas.
 - Comprensión de la arquitectura del software.
 - Control de versiones y documentación.
- **Actividades diarias:**
 - Codificación y depuración.
 - Revisiones de código.
 - Discusiones técnicas.

El **evaluador** tiene:

- **Responsabilidades principales:**
 - Diseño y ejecución de casos de prueba.
 - Identificación y documentación de errores.
 - Verificación de correcciones y realización de pruebas de regresión.
 - Garantizar que el software cumpla los requisitos y los estándares de calidad.
- **Tipos de pruebas:**
 - Funcionales.
 - De rendimiento.
 - De seguridad.
 - De aceptación del usuario.

El **analista funcional** debe / hace de:

- **Recopilar las necesidades del usuario:** realizar entrevistas para comprender necesidades y expectativas.

- **Traductor a requisitos:** convertir las necesidades empresariales en requisitos técnicos detallados para los desarrolladores.
- **Comunicación puente:** actuar como intermediario entre el equipo técnico y las partes interesadas en el negocio.
- **Documentar especificaciones:** crea documentación clara y detallada de los requisitos del sistema y las reglas de negocio.

El analista funcional es crucial para garantizar que el producto final aborde las necesidades reales del negocio.

El **líder del proyecto** tiene:

- **Responsabilidades claves:**
 - Coordinar los esfuerzos del equipo y asignar recursos.
 - Planificar el cronograma.
 - Monitorear el progreso.
 - Identificar y mitigar riesgos.
 - Informar el estado a las partes interesadas.
 - Resolver conflictos y eliminar obstáculos.

Los líderes de proyecto eficaces equilibran la comprensión técnica con las habilidades de gestión de personas, asegurando que tanto los resultados del proyecto como la dinámica del equipo se mantengan saludables.

El **usuario / cliente** tiene:

- **Definición de necesidades:**
 - Articula los problemas y oportunidades de negocio que el software debe abordar.
 - Proporciona contexto sobre procesos, limitaciones y objetivos.
- **Proporcionar retroalimentación:**
 - Revisar prototipos y primeras versiones para garantizar la alineación con las expectativas.
 - Sugiere ajustes y mejoras durante todo el desarrollo.
- **Validación de soluciones:**
 - Confirma si el producto entregado resuelve los problemas previstos y cumple con los requisitos.
 - Pruebas de aceptación final y aprobación.

Sin participación activa del cliente, incluso un software técnicamente perfecto puede no resolver la necesidad real del negocio.

En las metodologías ágiles, funciona de la siguiente forma:

Rol	Función Principal	Responsabilidades Claves
Product Owner	Representa los intereses comerciales	Prioriza las funciones, gestiona el backlog del producto (lista de tareas pendientes), maximiza el valor
Scrum Master	Facilitador de Proceso	Elimina los impedimentos, garantiza el cumplimiento de las prácticas ágiles, capacita al equipo
Equipo Ágil	Constructor del Producto	Colaboración interfuncional, progreso diario, entrega continua

Requerimientos

Un requisito es una condición específica que debe cumplir un sistema para satisfacer el contrato o la norma estipulada. Definen lo que el sistema debe hacer (funcionalidad) y cómo debe hacerlo (atributos de calidad).

Tipos de requerimientos:

- **Funcionales:** definen lo que debe hacer el sistema.
 - Características y capacidades.
 - Comportamientos específicos.
 - Procesos de entrada / salida.

Ejemplos: el sistema debe permitir a los usuarios registrarse con correo y contraseña, los usuarios deben poder restablecer sus contraseñas.

- **No funcionales:** definen cómo debe funcionar el sistema.
 - Criterios de rendimiento.
 - Medidas de seguridad.
 - Estándares de usabilidad.
 - Métricas de confiabilidad.

Ejemplos: el registro de usuario debe completarse en 2 segundos, el sistema debe ser compatible con todos los dispositivos móviles.

Errores comunes al momento de relevar requerimientos:

- **Ambigüedad:** los requisitos con múltiples interpretaciones conducen a malentendidos (ejemplo: el sistema debe responder rápidamente. ¿que significa rápidamente?).
- **Suposiciones no validadas:** dar por sentada información que no ha sido confirmada (ejemplo: asumir que todos los usuarios tienen conexiones a internet de alta velocidad).
- **Cambios no documentados:** los requisitos evolucionan, pero los cambios no se rastrean adecuadamente (ejemplo: acuerdos verbales para agregar funciones sin actualizar la documentación).
- **Combinar requisitos:** Interpretar múltiples requisitos como un único requerimiento (ejemplo: “la página de inicio de sesión debe usar tokens JWT y verse profesional”, plantear 2 requisitos por separado; “el sistema debe implementar JWT para la autenticación” y “la interfaz de usuario debe seguir las pautas de marca de la empresa”).

Objetivos y requerimientos

A la hora de definir requisitos, es indispensable consultar al cliente por sus necesidades.

La identificación de requisitos es una responsabilidad compartida entre el profesional y el usuario.

Los requisitos claros y bien definidos proporcionan la base para un desarrollo de software exitoso.

Dentro de los requerimientos, tenemos técnicas de relevamiento:

- **Entrevistas con las partes interesadas:** conversaciones individuales o grupales con usuarios y tomadores de decisiones para comprender las necesidades y expectativas.
- **Cuestionarios:** formularios estructurados para recuperar información esencial de las múltiples partes interesadas.
- **Análisis del sistema actual:** examinar soluciones actuales para entender las fortalezas, debilidades y oportunidades de mejora.
- **Prototipos:** representaciones visuales de la solución propuesta para validar la comprensión y asegurar retroalimentación de forma temprana.

Propuesta de proyecto

Es el documento fundamental que establece el marco para todo el proceso de desarrollo. Sirve como herramienta de comunicación inicial entre los desarrolladores y partes interesadas.

Las propuestas eficaces son concisas pero completas, demostrando viabilidad y evitando especificaciones técnicas abrumadoras.

Es importante porque sirve como brújula para el equipo de desarrollo, asegurando que todos comprendan la dirección y objetivos. Crea un entendimiento compartido entre el equipo de desarrollo y las partes interesadas para reducir malentendidos, y permite a los equipos evaluar la viabilidad técnica.

Componentes esenciales de una propuesta:

- **Descripción general del proyecto:** título memorable y descripción concisa de lo que se está construyendo.
- **Declaración del problema:** definición clara del problema que aborda el software.
- **Objetivos:** metas específicas y medibles que lograra el proyecto.
- **Definición del alcance:** define lo que será incluido y excluido de la solución.
- **Partes interesadas:** todas las partes involucradas en el proyecto.
- **Suposiciones y restricciones:** limitación y condiciones que afectan el desarrollo.
- **Criterios de aceptación:** cómo se medirá y evaluará el éxito del proyecto.

El **alcance** define los límites del desarrollo. Aclara lo que se entregara, y lo que no. Es la base para la asignación de recursos, tiempo, y gestión de riesgos. Representa el entendimiento entre los desarrolladores y las partes interesadas.

Es importante ya que previene malentendidos, establece criterios de éxito, permite una planificación precisa y facilita la negociación de funciones.

Las **inclusiones** del alcance definen lo que el equipo de desarrollo se compromete a entregar como parte del proyecto. Se aconseja expresar las inclusiones en lenguaje de alto nivel (ejemplo: “los usuarios pueden registrarse con verificación por correo electrónico” en vez de “el sistema implementa autenticación OAuth”).

Ejemplos de inclusiones: registro de usuarios, funcionalidad de copia de seguridad.

Las **exclusiones** declaran explícitamente lo que no se incluirá, evitando así la expectativa del cliente de algo que quizá da por asumido y puede no estar incluido.

Ejemplo de exclusiones: versiones de aplicación móvil, notificaciones por SMS.

La **tabla de definición de alcance** comunica lo que se entregará y lo que no, con contexto adicional para explicar las decisiones y establecer expectativas para futuras mejoras.

Funcionalidad	Incluída	Excluída	Observaciones
<i>Registro de usuario</i>	✓		Verificación de correo electrónico incluida
<i>Generación de informes PDF</i>	✓		Solo plantillas básicas
<i>Notificaciones de WhatsApp</i>		✓	Posible mejora futura
<i>Copias de seguridad automatizadas</i>		✓	Proceso de copia de seguridad manual documentado
<i>Integración de Mercado Pago</i>		✓	Se puede agregar en la versión 2.0
<i>Soporte multi-idioma</i>		✓	Solo interfaz en español
<i>Panel administrativo</i>	✓		Métricas básicas y gestión de usuarios

Las **partes interesadas** son personas, grupos u organizaciones que tienen interés o se ven afectadas por el proyecto de software. Pueden ser internas o externas.

Pueden tener diferentes niveles de influencia, distintas prioridades / necesidades, y pueden afectar el éxito o fracaso del proyecto. Cada parte interesada tiene un enfoque distinto de comunicación, diferentes niveles de detalle y lenguaje técnico.

Se clasifican de la siguiente manera:

- **Tomadores de decisiones:** clientes, ejecutivos y patrocinadores que aprueban presupuestos, cronogramas y cambios importantes.
- **Usuarios finales:** personas que interactuarán directamente con el software en sus actividades diarias.
- **Equipo de desarrollo:** profesionales responsables de construir la solución.
- **Autoridades académicas:** instructores o jefes que supervisan el proyecto.
- **Partes interesadas indirectas:** departamentos, partes o grupos afectados indirectamente por el sistema.

Es importante identificar las partes interesadas ya que permite establecer medios de comunicación, obtener información de requisitos, y comprender la perspectiva de cada parte individualmente.

La **matriz de poder / interés** ayuda a los equipos de desarrollo reconocer que tan influyente o partícipe es la parte interesada, para poder así asignar los recursos correspondientes.

El **poder** es la capacidad para influir en decisiones y recursos del proyecto.

El **interés** es cuánto se preocupa por los resultados del proyecto.

Ejemplos:

	Bajo Poder	Alto Poder
Bajo Interés	Monitorear (Comunicación mínima, ocasional)	Mantener satisfecho (Satisfacer sus necesidades, proporcionar actualizaciones a intervalos adecuados sin sobrecargar)
Alto Interés	Mantener informado (Comunicación regular, alentar participación)	Gestionar de cerca (Compromiso total, comunicación regular con información detallada)

Las **suposiciones** son condiciones que se consideran verdaderas, pero que no están garantizadas. Implican incertidumbre, deben ser documentadas y validadas, y requieren planificación de contingencia.

Las **restricciones** son limitaciones conocidas que deben respetarse. Son factores fijos, como restricciones de tiempo o técnicas, limitaciones de recursos, o requisitos legales / de cumplimiento. Se deben identificar temprano para crear planes realistas.

Los **criterios de aceptación** son condiciones específicas que deben cumplirse para que el cliente apruebe el producto final. Es un contrato claro entre el equipo de desarrollo y las partes interesadas.

La importancia de definirlos es:

- Eliminar la ambigüedad.
- Garantizar la calidad.
- Satisfacción del cliente.
- Claridad contractual.

Métricas - Estimaciones

La estimación es una habilidad fundamental que ayuda a establecer expectativas realistas y entregar con éxito.

Se deben realizar estimaciones para:

- **Anticipar el esfuerzo y la duración.**
- **Crear horarios realistas.**
- **Establecer las expectativas del cliente.**

Las estimaciones son aproximaciones fundamentadas. No son predicciones ni fruto de la adivinación. Proporcionan un marco de planificación razonable en vez de plazos exactos. Reducen el rango de incertidumbre a niveles manejables. Mejoran con la experiencia y datos históricos.

Una mala estimación puede tener distintas consecuencias:

- **Incumplimientos contractuales.**
- **Excesos de presupuesto.**
- **Conflictos con las partes interesadas.**

Hay distintas técnicas de estimación:

- **Juicio de expertos.**
- **Estimación por analogía.**
- **Descomposición de tareas.**

El **juicio de expertos** es una estimación basada en la experiencia personal o profesional de desarrolladores experimentados y líderes de proyectos. Es muy eficaz para equipos con amplia experiencia, y se basa en el conocimiento acumulado. Puede aplicar técnicas como **Delphi**, que se basa en una ronda de expertos sometidos a responder un cuestionario de forma anónima, que luego será analizado, generando un informe, para la posterior entrega de un segundo cuestionario el cual debe ser respondido en base a la retroalimentación recibida. El proceso se repite hasta llegar a un consenso.

La **estimación por analogía** consiste en comparar el proyecto actual con proyectos pasados similares para determinar estimaciones de tiempo basadas en el rendimiento histórico. Es ideal cuando la organización mantiene métricas detalladas de ciclos de desarrollo previos. Se debe tener en cuenta el alcance, la tecnología y composición del equipo, para llegar a una comparación justa.

La **descomposición de tareas** implica dividir proyectos grandes y complejos en componentes pequeños y manejables, para así estimarlos con mayor precisión. Se implementa identificando todos los componentes principales, se dividen en tareas específicas, se estima cada tarea individualmente, se suman para el tiempo total y se agregan contingencias para los desafíos de integración.

Dentro de la descomposición de tareas, tenemos técnicas de descomposición:

- **LOC:** por líneas de código. Se debe descomponer a fondo, llegando a considerables niveles de detalle.
- **PF:** por puntos de función. Se enfoca en cada características de dominio de información (entradas, salidas, consultas, interfaces).
- **PCU:** por puntos de casos de uso.

Se debe estimar un valor de tamaño optimista (**O**), más probable (**M**) y pesimista (**P**) para cada función. A partir de estos valores se puede aplicar la fórmula **PERT** para obtener el valor esperado.

$$Estimate = \frac{O + 4M + P}{6}$$

Como se puede ver, esta fórmula da mayor ponderación a la estimación más probable, al mismo tiempo que tiene en cuenta los escenarios de mejor y peor caso.

Las ventajas de aplicar la fórmula PERT son que se tiene en cuenta la incertidumbre, reduce el impacto de las opiniones extremistas y proporciona un enfoque ponderado basado en la probabilidad. Ejemplo:

$$Estimate = \frac{4 + 4 \times 8 + 20}{6} = \frac{4 + 32 + 20}{6} = \frac{56}{6} = 9.3 \text{ horas}$$

La fórmula básica para la estimación de costos es:

$$\text{Costo Total} = (\text{Esfuerzo Estimado} \times \text{TarifaPorHora}) + \text{Costos Adicionales}$$

El **esfuerzo estimado** es la cantidad de horas - hombre requeridas. El resto se decantan por su nombre.

Los costos adicionales pueden incluir licencias de software, infraestructura, mantenimiento.

Módulo	Hs. (PERT)	Perfil Desarrollador	Tarifa	SubTotal
Registro de Usuarios	10	Junior	\$300	\$3000
Generación de Informes	15	Semi Senior	\$500	\$7500
Integración de Pagos	20	Senior	\$800	\$16000
Panel de Usuarios	12	Semi Senior	\$500	\$6000
Migración de Base de datos	8	Senior	\$800	\$6400
Costos Adicionales (licencias, alojamiento, etc.)				\$5000
Costo Total de Proyecto				\$43900

La **subestimación** puede traer problemas de calidad, agotamiento de desarrolladores y posible fracaso de proyecto.

La **sobreestimación** puede traer satisfacción del cliente, posible ampliación del alcance, utilización ineficiente de recursos y mala prensa.

Es importante realizarse algunas preguntas en base a la estimaciones realizadas, para determinar si se cayó en subestimaciones o sobreestimaciones, y también para anticiparse a cuestionamientos del cliente al recibir la cotización.

Algunas preguntas que se podrían realizar:

- ¿Qué función fue la más costosa? ¿Por qué? ¿Cómo se podría optimizar?
- ¿Qué perfil de desarrollador se utilizó mayormente? ¿Es eficiente para este tipo de proyecto?
- ¿Qué costos fueron inesperados o se pasaron por algo? ¿Cómo podría predecirse en el futuro?